VehicleGPT Edge

Evidence-Grounded Vehicle Intelligence on Constrained Hardware

CASE STUDY 5 — TECHNICAL PROPOSAL

Natural Language Vehicle Information using On-Device SLM/LLM

Status: Proposed architecture — not an implemented or benchmarked system

Date: 25 October 2026

Design principle: retrieve and calculate locally; use the model to explain evidence; keep vehicle control isolated.

Document guide

1. Executive summary

2. Problem definition and scope

3. Goals, non-functional requirements and success criteria

4. High-level architecture

5. Vehicle data collection strategy

6. Simulation and reproducible scenarios

7. Data model and historical storage

8. SLM/LLM selection and edge runtime

9. Natural-language query processing

10. Search, retrieval and temporal reasoning

11. Trends, anomalies and explanation policy

12. Edge deployment and operating system

13. User interaction design

14. Privacy, security and safety isolation

15. Evaluation and benchmark plan

16. Implementation roadmap

17. Deliverables and demo plan

18. Risks, limitations and mitigations

19. Conclusion

References and verification notes

Conventions: official requirement = from the supplied brief; design decision = proposed architecture; target = must be measured; limitation = not claimed as solved.

1. Executive summary

VehicleGPT Edge is a proposed offline-first vehicle information assistant. It lets a driver, service engineer, validation team, or fleet operator ask natural-language questions about vehicle signals, diagnostic events, ADAS events, and historical driving behaviour. It combines deterministic data retrieval and analytics with a compact on-device Small Language Model (SLM) that explains retrieved evidence.

The central engineering decision is to keep measurements and calculations outside the language model. A query router selects a safe, typed execution plan; SQL and analytics compute values; temporal logic reconstructs event sequences; and the model receives a bounded evidence package. A validator checks units, numbers, timestamps, and source references before an answer is shown.

The official brief sets the key targets: no cloud dependency, approximately 2 GB RAM and about two ARM A5X-class cores, common-query response under two seconds, searchable history spanning several weeks, and strict isolation from safety-critical vehicle functions. These are design targets to verify on a chosen board—not results already achieved.

2. Problem definition and scope

Software-defined vehicles generate data across sensors, ECUs, ADAS, powertrain, body electronics, infotainment, diagnostics, and logs. Finding a trustworthy answer often requires joining multiple signals and event histories over time. Raw logs are difficult to query manually, while a cloud assistant conflicts with the brief's offline and privacy requirements.

The system must answer questions such as: why range changed over two weeks; when efficiency dropped most; which ADAS disengagement events occurred yesterday; what preceded a battery-overheating warning; and how city and highway energy consumption compare.

In scope	Out of scope for the initial prototype
Read-only ingestion, historical search, trends, event timelines, explainable summaries, offline local UI	Autonomous vehicle control, actuator commands, remote/cloud inference, replacing certified diagnostics, claiming causation from correlation alone
Simulated and file-based adapters first; protocol adapters designed behind interfaces	Assuming every listed automotive protocol is physically connected in the demo

The first release should be a demonstrable engineering prototype using reproducible simulated/recorded data. Production integration requires OEM-specific signal dictionaries, access controls, hardware validation, cybersecurity review, and safety analysis.

3. Goals, non-functional requirements and success criteria

Requirement	Design response	Acceptance evidence
Offline operation	All query, retrieval and inference paths run locally	Disable network; demonstrate successful queries and no outbound traffic
~2 GB RAM / ~2 ARM cores	Quantized compact model; short context; model-free fast path; bounded caches	Peak RSS and latency measured on reference target
<2 seconds for common queries	Typed SQL templates, indexes, cached metadata, small result sets; generation only when needed	p50/p95 latency from at least 30 runs per query class
Several weeks of history	Time-partitioned/retained SQLite records, indexes, rollups	Query ground-truth scenarios across at least 21–30 days of generated data
Multiple sources	Adapter interface and canonical signal/event schema	Replay a fixture from at least two source types
Safety isolation	Read-only interfaces; no actuator API or control bus write permission	Architecture review and negative tests for write paths
Trustworthy answers	Evidence package, provenance, missing-data handling and validation	Numeric/temporal answer accuracy against ground truth

Success criteria are intentionally measurable. A benchmark cell must remain 'not measured' until the system is actually run on the named device and software version.

4. High-level architecture

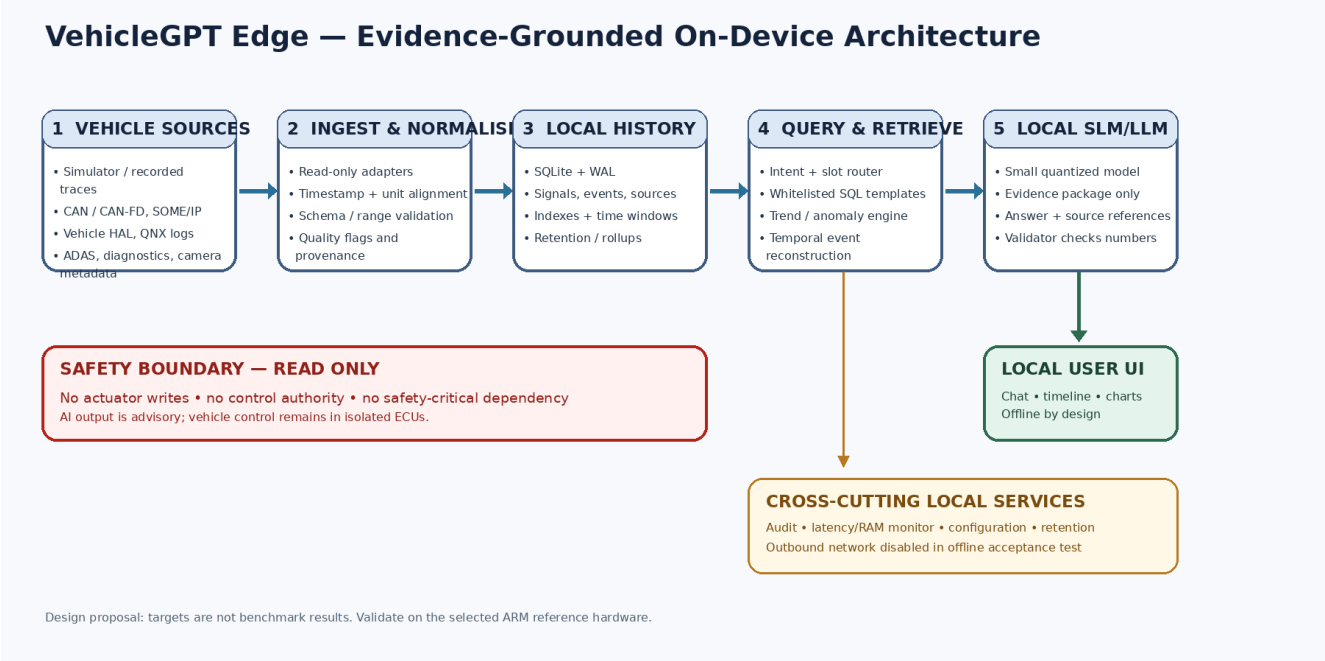


Figure 1. Proposed offline-first architecture. Safety-critical functions remain isolated.

Data flow: (1) a simulator or source adapter emits timestamped observations/events; (2) ingestion validates, normalises units and attaches provenance; (3) the local store retains raw/aggregated history; (4) a query router maps the user's question to a typed intent and parameters; (5) deterministic SQL, analytics and temporal search create an evidence package; (6) a local SLM turns that package into a concise explanation; (7) a validator checks the response before the local UI displays it.

Component	Responsibility
Source adapters	Isolate protocol/file/simulator specifics from the canonical data model
Normaliser and quality checks	Convert timestamps/units, reject malformed records, flag missing or implausible values
Local data store	Store signals, events, source metadata, rollups and audit records
Query router	Classify intent, extract time range/signal/vehicle/scenario slots, choose a typed plan
Analytics and temporal engine	Compute aggregates, change points, threshold events, and ordered event windows
Evidence package	Carry result values, units, timestamps, source IDs, quality flags and calculation method
Local SLM/LLM	Explain evidence in plain language; do not invent measurements or execute arbitrary tools
Answer validator	Verify numeric values, citations, units and uncertainty statements
Local UI and operations	Show chat, timeline, charts, audit, latency, RAM and ingestion status

The safety boundary is architectural, not merely a prompt instruction. The AI service has no actuator-write API, no ability to modify safety-critical ECU settings, and no authority to alter vehicle operation. It is an information and diagnostic-support layer.

5. Vehicle data collection strategy

Use a canonical adapter contract: each adapter emits SignalSample or VehicleEvent records with event time, ingestion time, vehicle/session ID, source, signal/event name, value, unit, quality status and provenance. Keep raw source identifiers so an engineer can trace a normalized record back to its origin.

Source	Collection approach	Prototype maturity
CAN / CAN-FD	Read-only SocketCAN or recorded BLF/ASC/CSV replay; decode with a versioned DBC where available	File replay first; live bus integration later
SOME/IP services	Read-only service/event adapter; document service IDs, event groups and schema/version	Simulated adapter unless an actual service is available
Android Vehicle HAL	Adapter to permitted vehicle properties/events through the relevant Android Automotive service boundary	Mock interface or Android Automotive emulator

Source	Collection approach	Prototype maturity
QNX services/logs	Read-only log import/forwarder with source timestamps and severity metadata	Recorded log fixture first
ADAS and camera feeds	Prefer structured ADAS events and camera metadata; avoid storing raw video unless needed	Event fixture; optional simulator output
Diagnostics	Import DTC/UDS/OBD-derived records with timestamp, code, status and source	Synthetic diagnostic events initially
Simulator	Scenario generator emits ground-truth signals/events at controlled intervals	Required demo source

Adapters should not assume universal signal naming or units. Maintain a versioned signal dictionary (e.g., vehicle_speed_kph, battery_temp_c, soc_pct, energy_wh_per_km) and explicit conversion rules. If a unit, timestamp or mapping is unknown, preserve the raw field and flag it rather than silently guessing.

For the demo, begin with CSV/JSON replay and a simulator adapter. That gives reproducible tests and does not imply that live CAN, QNX or SOME/IP integration has been completed.

6. Simulation and reproducible scenarios

The official brief explicitly asks how driving scenarios and error sequences will be recreated. A full CARLA integration is optional; a lightweight deterministic simulator is the fastest way to establish ground truth. An optional CARLA adapter can later add driving context, while the canonical schema remains unchanged.

Scenario	Generated sequence	Expected query answer
Range reduction over 14 days	Lower average SOC-adjusted distance, increased auxiliary load, varied ambient temperature and trip mix	Show range/efficiency trend, compare confounders and report the evidence; do not assert one cause unless data supports it
Largest efficiency drop	Trip-level Wh/km values with one controlled step change	Return time window, baseline, post-change value and percentage delta
ADAS disengagement yesterday	Timestamped disengagement events with reason/status when available	List events in the requested local-time window with source and quality
Battery overheating warning	Temperature rise, warning threshold event, cooling state and preceding load	Reconstruct ordered events before the warning and distinguish temporal order from causality
City vs highway energy	Trips tagged by drive cycle and distance/energy	Compare Wh/km, sample counts and variability for each group
Missing/stale sensor data	Dropouts, duplicate records, clock skew and impossible values	State data gaps and reduce confidence instead of fabricating continuity

Each scenario should have a fixed seed, a generated dataset, a ground-truth JSON file, and a test script. The demo must replay the same scenario on demand so reviewers can reproduce the same answers.

7. Data model and historical storage

Use SQLite for a compact, dependable first implementation. WAL mode allows readers to access a consistent snapshot while ingestion writes, but concurrency and durability settings must be tested for the chosen device and filesystem. A time-series database is not required for the first prototype.

Table	Key fields and purpose
vehicle_signals	id, vehicle_id, event_ts, ingest_ts, signal_name, value_num, unit, source_id, quality_flag, session_id
vehicle_events	id, vehicle_id, event_ts, event_type, severity, code, description, source_id, session_id
data_sources	source_id, adapter_type, source_name, schema_version, trust_level, last_seen_ts
signal_dictionary	signal_name, canonical_unit, description, plausible_min/max, mapping_version
query_audit	query_id, timestamp, intent, duration_ms, model_used, result_count, validation_status
rollups	window_start, window_end, signal_name, aggregation, value, sample_count, quality_summary

Index the signal table on (vehicle_id, signal_name, event_ts) and event table on (vehicle_id, event_type, event_ts). Use parameterized SQL, bounded query windows, and pagination. For recurring analytics, compute rollups at ingestion or scheduled maintenance rather than rescanning every raw row for every question.

Storage sizing must be measured from the chosen sampling rate and schema. For a rough upper-bound illustration, 20 signals/second × 86,400 seconds/day × 30 days = 51.84 million signal samples. At that scale, record width, indexes, write amplification and query plans matter. The prototype should therefore support configurable sampling/aggregation, retention policies, and a measured storage budget.

8. SLM/LLM selection and edge runtime

Model choice is a benchmark decision, not a brand preference. The runtime must load locally, support quantized weights, run on the target CPU/OS, fit the memory budget together with the OS/database, and produce acceptable explanations for the fixed test suite. The primary path should not require generation for exact lookups or simple aggregates.

Candidate	Why test it	Risks / checks
Qwen2.5-0.5B-Instruct (quantized)	Small fallback profile for the strictest memory budget	May struggle with multi-step explanation; test routing and grounded summaries
Qwen2.5-1.5B-Instruct (quantized)	Proposed primary candidate balancing size and instruction-following	Must measure peak RSS, load time and response latency on target
Llama 3.2 1B Instruct	Alternative compact instruction model	Review model licence and distribution conditions
SmolLM2-1.7B-Instruct	Compact comparison candidate	Validate tool-free summarisation, quality and memory use

Proposed runtime: llama.cpp with a compatible GGUF quantization, initially Q4_K_M where available. Keep the context window small (e.g., 2k–4k tokens for the strict profile), cap output tokens, and feed only a compact evidence package. Ollama can be convenient during laptop development, but the final constrained deployment should be benchmarked with the actual runtime and packaging intended for the board.

A 1.5B-parameter model at roughly 4–5 bits per parameter has a raw weight footprint around the order of 1 GB before runtime buffers, KV cache, allocator overhead and OS memory. This is an estimate, not a device measurement. Use a smaller 0.5B model or a model-free answer path if the complete process exceeds the RAM budget.

Selection protocol: use the same 50–100 query set and identical evidence packages for each candidate; record exact-match/numeric correctness, grounding, refusal-to-guess behaviour, p50/p95 latency, time to first token, peak RSS and energy/CPU where measurable. Check the actual licence for the exact model revision before redistribution.

9. Natural-language query processing

The language model should not produce arbitrary SQL. Parse each question into a typed query plan with a finite set of intents, validated slots, allowed signal names, bounded time ranges and approved operations. Use deterministic templates for execution.

Intent	Example	Execution
Direct lookup	What is the current battery temperature?	Latest valid value query
Trend / comparison	Compare city and highway energy consumption	Grouped aggregation with sample counts and units
Event search	Show ADAS disengagement events yesterday	Event-type filter and timezone-aware interval
Temporal reconstruction	What happened before the overheating warning?	Target event plus configurable preceding window and ordered signals/events
Explanation	Why did range drop?	Retrieve relevant trends and candidate correlates; explain evidence and caveats

Pipeline: normalize question → detect intent → extract entities/time range → resolve aliases against signal dictionary → validate slots → execute query template → package evidence → optionally generate explanation → validate output → render citations. If the router confidence is low, ask a clarifying question or show the best-supported interpretation.

Use a model-free fast path for common queries. Only invoke the SLM when the answer benefits from natural-language synthesis. This reduces latency and avoids introducing language-model variability into numerical calculations.

10. Search, retrieval and temporal reasoning

Structured SQL is the primary retrieval method because vehicle signals and events have timestamps, units and known schemas. Vector search is optional for unstructured service manuals or diagnostic documentation; it should not replace SQL for exact measurements.

A temporal query is a sequence reconstruction problem: locate the anchor event; retrieve signals/events in a bounded time window before and after it; align timestamps; sort by event time; mark missing intervals; and summarize observed changes. Do not equate ‘occurred before’ with ‘caused by’.

Evidence package fields: query ID; selected intent and time window; rows/aggregates; units; calculation formulas; source IDs; quality flags; sample counts; missing-data notes; and links to the relevant timeline. The SLM is instructed to use only this package for vehicle-specific claims.

For explanations such as range reduction, compare candidate factors—trip mix, ambient temperature, auxiliary loads, speed profile, tire/maintenance data if available—and report which are supported, unsupported or absent. If the needed signal was not collected, explicitly state that the cause cannot be determined from current data.

11. Trends, anomalies and explanation policy

Start with transparent, testable analytics before adding learned anomaly models. Useful initial methods include threshold rules, rolling means, percentage deltas, robust z-scores, change-point detection and event-window correlation. Record the formula, sample size and threshold used.

For a numeric series x, a simple baseline is the rolling mean μ and standard deviation σ over a defined historical window. A candidate anomaly score is $z = (x - \mu) / \sigma$; if σ is near zero or the window is too small, do not use the score. Robust median/MAD methods may be preferable when outliers distort the mean.

Explainability policy: report the observed change, the relevant time interval, supporting signals/events, data quality, and plausible hypotheses. Use ‘coincided with’ or ‘is consistent with’ unless a controlled scenario or stronger evidence supports causal attribution. Show confidence as evidence quality, not a fabricated probability.

12. Edge deployment and operating system

Development can start on a laptop and then move to an ARM reference device or emulator. Keep the runtime portable: Python for orchestration and analytics; SQLite for storage; a local inference runtime; a small local HTTP/UI layer bound to loopback; and system services for ingestion and retention.

Profile	Purpose	Configuration
Developer profile	Fast iteration on laptop	Full logs, test data, optional Ollama for experimentation, repeatable pytest suite
Constrained reference profile	Test the stated resource limits	Quantized small model, bounded context, limited workers, SQLite cache cap, no unnecessary services
Production integration profile	Future OEM/vehicle integration	Approved OS/runtime, signed packages, hardware security controls, OEM interfaces, threat model and validation

The official brief mentions automotive ARM boards, Raspberry Pi 4/5 with an AI HAT, ARM virtualized environments such as Cuttlefish/QEMU, and equivalent platforms. Choose one target and state exact RAM, CPU, OS image, kernel, runtime version and power mode in every benchmark. Raspberry Pi is a demonstrator, not proof of automotive-grade qualification.

Resource controls: cap model context and output; bound SQLite caches and result sizes; avoid multiple model instances; pre-load the model only if the memory budget allows; expose a model-free fallback; and monitor peak resident set size. Do not claim the <2-second target until measured on the intended target.

13. User interaction design

Use a local web UI that works without internet. The main screen should provide a natural-language question field, vehicle/session selector, time range, answer with evidence links, and a timeline/chart view. Each answer should disclose the data window, source coverage, sample count, and missing data.

View	Required elements
Overview	Vehicle/session status, last ingestion time, storage window, offline status, RAM and latency indicators
Ask VehicleGPT	Question, answer, evidence cards, source references, uncertainty/missing-data notes
Timeline	Chronological events and signal changes around a selected event
Analytics	City/highway comparisons, trends, threshold events and sample counts
Performance / audit	Query latency, peak RSS, model path used, validation status, source health

Use a clear distinction between a measured fact, a computed metric, a hypothesis, and a missing-data limitation. Avoid presenting an AI-generated explanation as a confirmed diagnosis.

14. Privacy, security and safety isolation

Local-only means the default runtime must not depend on a cloud API, remote vector database, analytics beacon or model download at query time. Model files and updates may be provisioned during controlled setup, but the application should be able to operate with network interfaces disabled.

Security controls: least-privilege service account; read-only vehicle interfaces; no arbitrary shell tools; parameterized SQL; allowlisted query templates; local authentication if multiple users exist; filesystem permissions; encrypted storage where supported; redacted logs; signed/versioned model and signal dictionaries; and a documented retention/delete policy.

Treat imported logs and retrieved text as untrusted input. They must not be allowed to override system instructions or trigger tool execution. The model has no unrestricted SQL, shell, network or actuator tools. Validate generated claims against the evidence package.

Safety rule: the assistant is advisory and isolated. It must never command steering, braking, powertrain, battery-management or ADAS control. Production use would require an automotive cybersecurity threat model, integration testing and applicable safety/process reviews; this proposal does not claim certification.

15. Evaluation and benchmark plan

Run a fixed benchmark suite with ground truth. Separate functional accuracy from runtime performance, and test both the deterministic fast path and model-assisted path. Record hardware/software configuration and raw results so the evaluation can be reproduced.

Metric	Definition / method	Target / status
Intent routing accuracy	Correct intent classifications / total test questions	Target to define; measure
Numeric correctness	Numeric answers within declared tolerance of ground truth	Measure; no result claimed
Temporal accuracy	Correct event set and order for timeline questions	Measure
Evidence grounding	Vehicle-specific claims supported by retrieved evidence	Aim for 100% of numeric claims to map to evidence
Common-query latency	Monotonic-clock end-to-end latency; p50/p95, ≥30 repeats	Official target <2 s for common queries; measure on target
Peak memory	Peak RSS or cgroup memory.peak during representative queries	Fit the selected ~2 GB system budget; measure
History coverage	Successful query over generated 21–30 day history	Several weeks searchable; verify
Offline test	Run with network disabled and inspect outbound connections	No cloud dependency
Security tests	SQL injection, prompt injection, malformed logs, attempted write paths	No unsafe execution; document results

Report benchmark values only after running the test. The submission should include a results table with hardware, OS, runtime, model revision/quantization, context length, dataset seed, repetitions, median, p95 and pass/fail. Keep unmeasured fields explicitly blank or ‘not measured’.

16. Implementation roadmap

Phase	Work	Exit criterion
1 — Data contract	Define canonical signals/events, units, source metadata and quality flags	Schema tests pass; documented field dictionary
2 — Simulator	Generate six reproducible scenarios with seeded ground truth	Same seed yields same data and expected answers
3 — Storage	Implement SQLite schema, indexes, WAL and retention/rollups	21–30 day dataset loads and indexed queries work
4 — Deterministic query engine	Typed intent router and parameterized SQL templates	Five required example questions answered without an LLM
5 — Temporal analytics	Event windows, trend deltas, city/highway grouping, anomaly rules	Ground-truth scenario tests pass
6 — Local SLM	Integrate quantized model and bounded evidence prompt	No invented numeric values in test suite
7 — UI and audit	Chat, timeline, charts, source references, performance metrics	Offline end-to-end demo
8 — Edge profiling	Deploy to named ARM/reference target; measure RSS and latency	Publish reproducible benchmark table
9 — Security review	Test untrusted logs, SQL injection, offline behaviour and read-only boundary	Threat model and test evidence
10 — Packaging	Setup guide, sample data, model licence notes, known limitations	Fresh-machine install and repeatable demo

Recommended build order: do not begin with the LLM. First make the data pipeline, storage, query templates and ground-truth tests correct. Then add local generation as a presentation layer over verified evidence.

17. Deliverables and demo plan

Technical deliverables: architecture diagram; source-adapter contract; simulator and six scenario fixtures; local schema and sample history; typed query router; SQL/analytics/temporal engine; local SLM runtime configuration; local UI; benchmark scripts and blank/results tables; threat model; setup guide; model/licence inventory; and known-limitations statement.

Suggested 4–5 minute demo: (1) show offline status and source coverage; (2) ask for the largest efficiency drop; (3) show the computed result and supporting time window; (4) ask what happened before a battery warning and display the timeline; (5) compare city/highway energy; (6) disable networking and repeat a query; (7) show latency/RAM and state whether the target is measured or still pending.

18. Risks, limitations and mitigations

Risk	Mitigation
2 GB memory is too tight for the chosen model	Use 0.5B fallback or model-free fast path; reduce context; measure peak RSS
Language model hallucinates a cause	Evidence-only prompt, explicit uncertainty, answer validator, no causal language without support
Signals differ across vehicle platforms	Versioned signal dictionary and adapters; preserve raw fields; unknown-unit flags
Historical queries become slow	Indexes, rollups, bounded windows, query-plan inspection and load tests
Simulator appears unrealistic	Publish scenario assumptions and ground truth; add recorded/CARLA traces when available
Privacy leakage through logs or dependencies	Offline test, least privilege, local-only dependencies, retention and redaction
False confidence in automotive readiness	Label as prototype; no safety-control path; separate demo board from automotive-grade qualification

Known limitations of the initial design: simulated data cannot prove real-vehicle integration; correlation cannot prove causation; compact SLM quality may be limited; resource performance is hardware-dependent; and production vehicle deployment needs OEM-specific integration, cybersecurity and safety validation.

19. Conclusion

VehicleGPT Edge is designed as a local evidence-retrieval and explanation system rather than a free-form chatbot. Its strongest differentiator is a verifiable chain from natural-language question to typed query, computed result, source evidence and validated explanation. This architecture directly addresses offline operation, historical search, constrained hardware and safety isolation while leaving model selection and performance as measurable engineering decisions.

References and verification notes

Primary source: Case Study 5 — Natural Language Vehicle Information using On-Device SLM/LLM (VehicleGPT), supplied competition brief. The brief is authoritative for the challenge requirements and evaluation criteria.

Technical references to verify before final submission: llama.cpp project documentation and repository; SQLite Write-Ahead Logging documentation; Qwen2.5 and SmolLM2 model cards and exact model licences; Meta Llama 3.2 model licence; Linux SocketCAN documentation; AUTOSAR SOME/IP materials or vsomeip documentation; Android Automotive Vehicle HAL documentation; CARLA simulator documentation; Raspberry Pi 5 product specifications; and applicable OEM/ISO/AUTOSAR documentation for any real integration.

Important: model names and runtime options above are candidates, not a claim that a particular model has passed the stated memory/latency limits. Record the exact revision, quantization, licence, hardware, software versions and measured results before presenting final performance claims.